

Sistemas Operativos

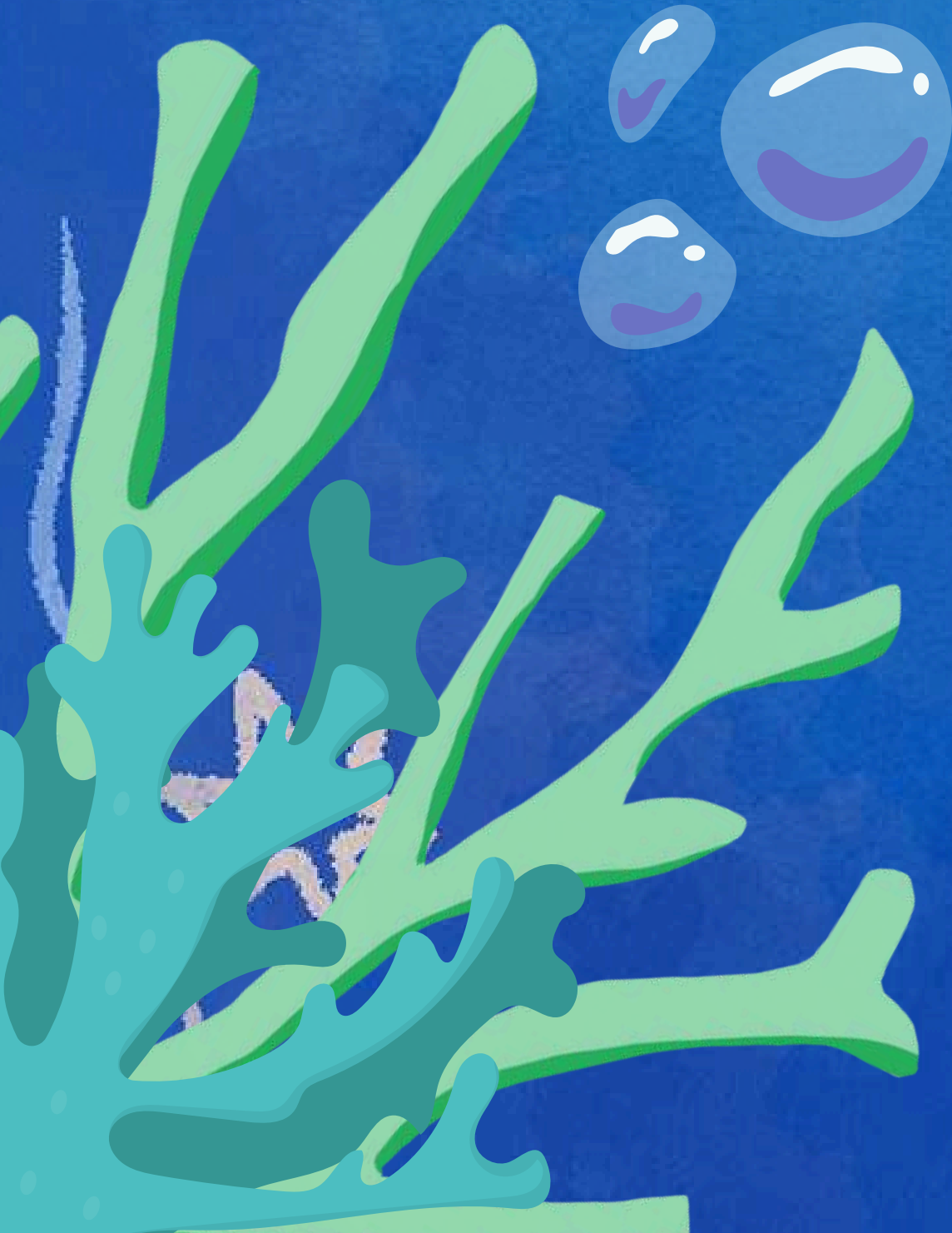
Sistemas Operativos en Equipos Nauticos

POR:

Martinez Garcia Luis Angel
Urbano Meza Joseph Gael

Índice

1. Introducción
2. Fundamentos de los Sistemas Operativos
3. ¿Cómo funcionan?
4. ¿Para qué se utilizan?
5. Requisitos Especiales en el Entorno Náutico
6. RTOS
7. Tipos y Ejemplos de Sistemas Operativos en Equipos Náuticos
8. Aplicaciones Prácticas a Bordo
9. Estándares y Seguridad Marítima
10. Casos Reales y Tecnologías Emergentes
11. Retos y Futuro de los SO en la Industria Náutica
12. Conclusiones.





Introducción

El mundo náutico ha experimentado una transformación tecnológica radical en las últimas décadas. Desde los tradicionales sextantes y brújulas, hasta depender de complejos sistemas informáticos que permiten una navegación precisa, segura y automatizada. La pieza fundamental para que esto pueda suceder se encuentran en los Sistemas Operativos (SO), piezas fundamentales que gestionan el hardware y el software de los equipos náuticos, permitiendo un funcionamiento coordinado y eficiente.

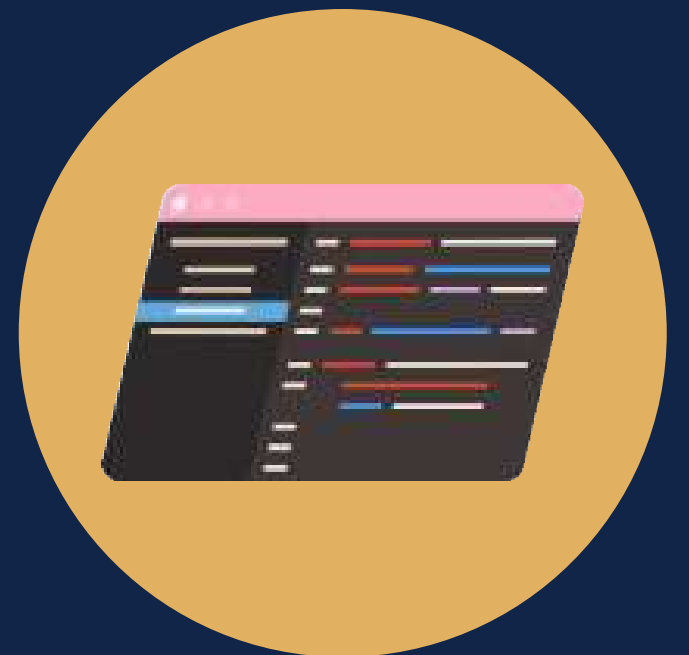


Fundamentos de los sistemas operativos

Un sistema operativo es el software principal que gestiona los recursos de un sistema informático. Actúa como intermediario entre el hardware y el software de aplicación, permitiendo que múltiples tareas se ejecuten eficientemente.

En contextos náuticos, estos sistemas pueden encontrarse en:

- Equipos de navegación.
- Controladores de motor.
- Sistemas de comunicación y monitoreo ambiental.



¿Cómo funcionan?

El funcionamiento de los SO se basa en la gestión eficiente y continua de los recursos electrónicos a bordo de una embarcación.

- Gestión del Hardware Náutico: el sistema operativo se encarga de controlar y coordinar diversos dispositivos.
- Interfaz de Usuario: el sistema operativo también gestiona la interfaz gráfica con la que interactúa el personal de la embarcación.
- Comunicación entre Sistemas: el sistema operativo facilita la interconexión entre diferentes módulos del barco.
- Procesamiento en Tiempo Real: muchos de estos sistemas requieren procesamiento inmediato de datos para garantizar decisiones rápidas y seguras.
- Gestión de Recursos y Seguridad: el SO administra recursos como la memoria, energía, redes de datos y almacenamiento.
- Actualización y Diagnóstico.



¿Para qué se utilizan?

Los sistemas operativos náuticos son esenciales para:

- Navegación automática y asistida (autopiloto, planificación de rutas).
- Control de motores y propulsión.
- Gestión energética (sistemas eléctricos, combustible).
- Monitoreo ambiental (clima, profundidad, temperatura).
- Seguridad y comunicación (AIS, VHF, ECDIS, sistemas de emergencia).
- Diagnóstico y mantenimiento remoto.



Requisitos Especiales en el Entorno Náutico

Tiempo real

Respuestas inmediatas a eventos críticos (alarma de colisión, fallos en motor).



Tolerancia a fallos:

Sistemas redundantes para evitar pérdida de funciones clave



Cumplimiento normativo:

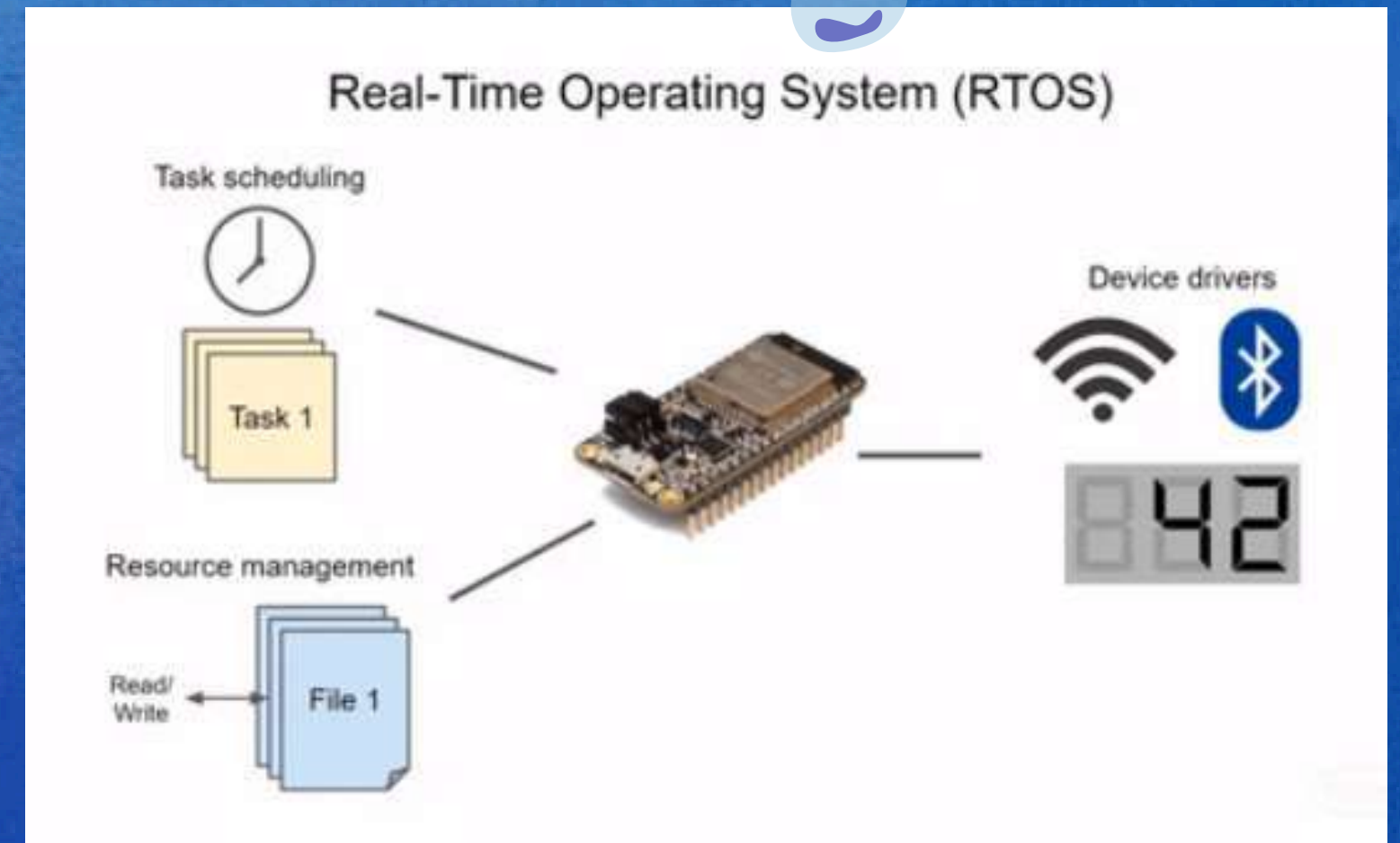
Certificaciones como SOLAS, ISM Code, y requerimientos de la OMI.



Sistemas Operativos en Tiempo Real (RTOS)

Un sistema operativo en tiempo real es muy similar a un sistema operativo de propósito general, pero se diferencian principalmente en tres puntos:

- Primeramente se encarga de ejecutar procesos o tareas, asignando un tiempo de ejecución dependiendo de su prioridad.
- Gestiona archivos y carpetas a los que tienen acceso las aplicaciones.
- Por último, gestiona los controladores de los dispositivos conectados, como teclados, tarjetas de sonido, discos duros, etc.



Sistemas Operativos en Tiempo Real (RTOS)



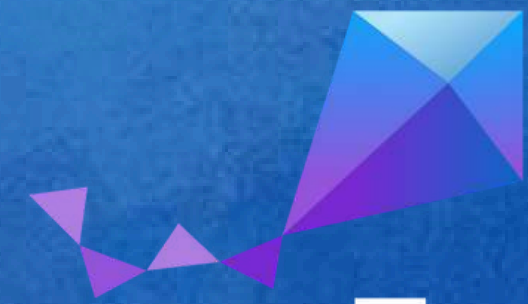
VxWorks

SO en tiempo real muy utilizado en navegación y control de propulsión.



QNX

Popular en sistemas críticos por su arquitectura de microkernel y alta fiabilidad.



Zephyr®

Zephyr

Ligero, modular y de código abierto. Además de ser usado en sensores y dispositivos IoT marítimos.



VxWorks

VxWorks es un sistema operativo en tiempo real (RTOS) desarrollado por la empresa Wind River Systems. Es ampliamente utilizado en sistemas embebidos que requieren alta confiabilidad, bajo consumo de recursos y respuesta determinista, especialmente en aplicaciones críticas como la industria aeroespacial, automotriz, militar y náutica.

Fue lanzado por primera vez en 1987 y ha sido usado en misiones espaciales (como el Mars Rover), controladores industriales, aeronaves, y sistemas de navegación marítima.

WINDRVR

VxWorks

Características claves:

- Tiempo real.
- Determinismo.
- Multitarea con planificación por prioridades.
- Alta fiabilidad y estabilidad.
- Soporte para múltiples arquitecturas.
- Seguridad y tolerancia a fallos.
- Soporte para redes, archivos, drivers y gráficos embebidos.

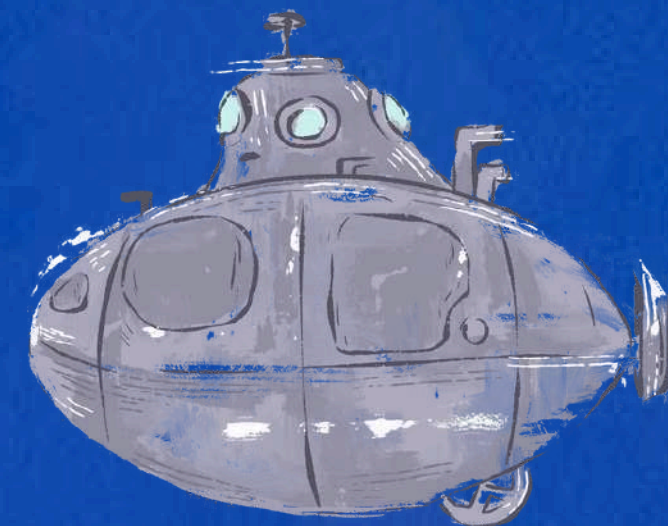
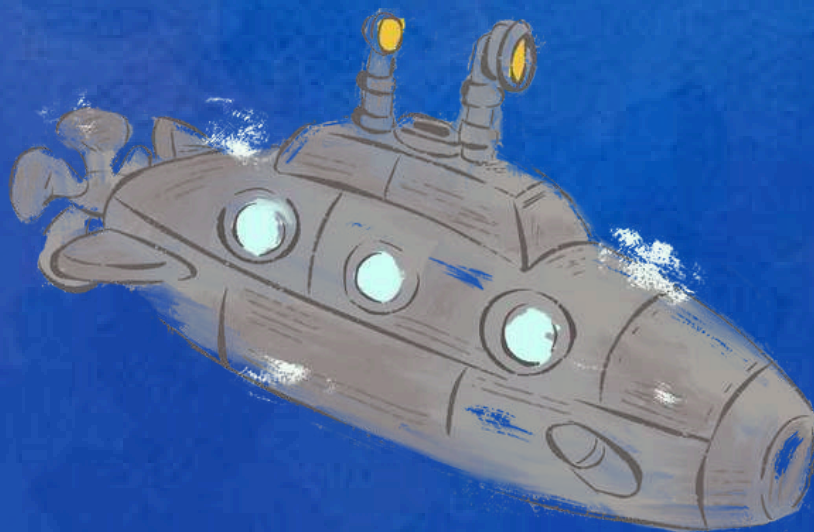
Usos:

- Navegación automática y radares.
- Control de motores y propulsión.
- Sistemas de armas navales o vigilancia en buques militares.
- Drones acuáticos y submarinos autónomos (AUVs).
- Sistemas de posicionamiento dinámico (DP) en plataformas petroleras marinas.

VxWorks

Ventajas:

- Funciona bien en condiciones extremas (sal, vibración, humedad).
- Permite una respuesta rápida ante emergencias (maniobras automáticas, alarmas).
- Soporta integración con sensores y protocolos náuticos (como NMEA 2000).
- Alta estabilidad: puede operar meses o años sin reinicios.





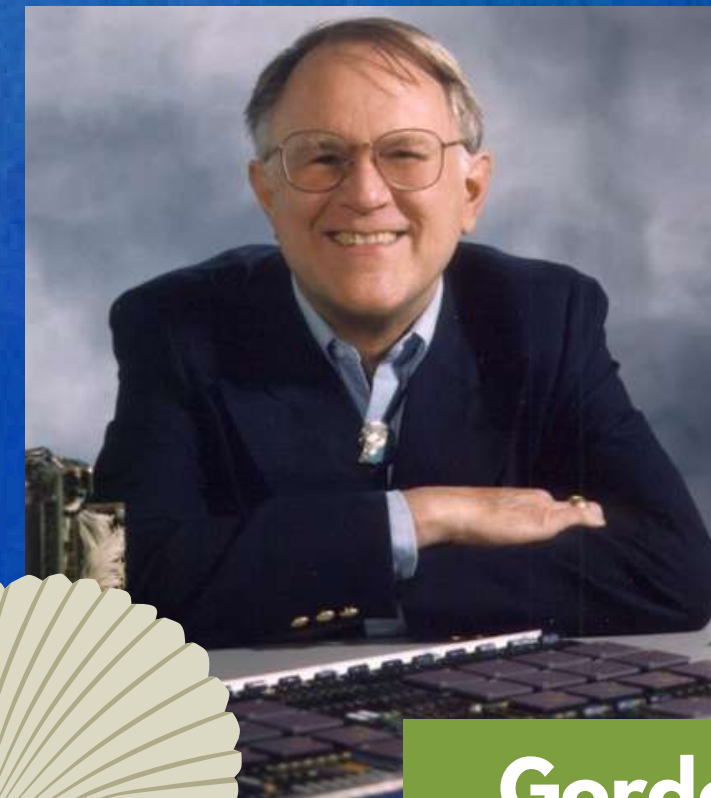
QNX está basado en una estructura de micronúcleo, que proporciona características de estabilidad avanzadas frente a fallos de dispositivos, aplicaciones, etc.

El sistema operativo QNX fue fundado por Gordon Bell y Dan Dodge es una plataforma de OS de tiempo real que incorpora la arquitectura UPM.

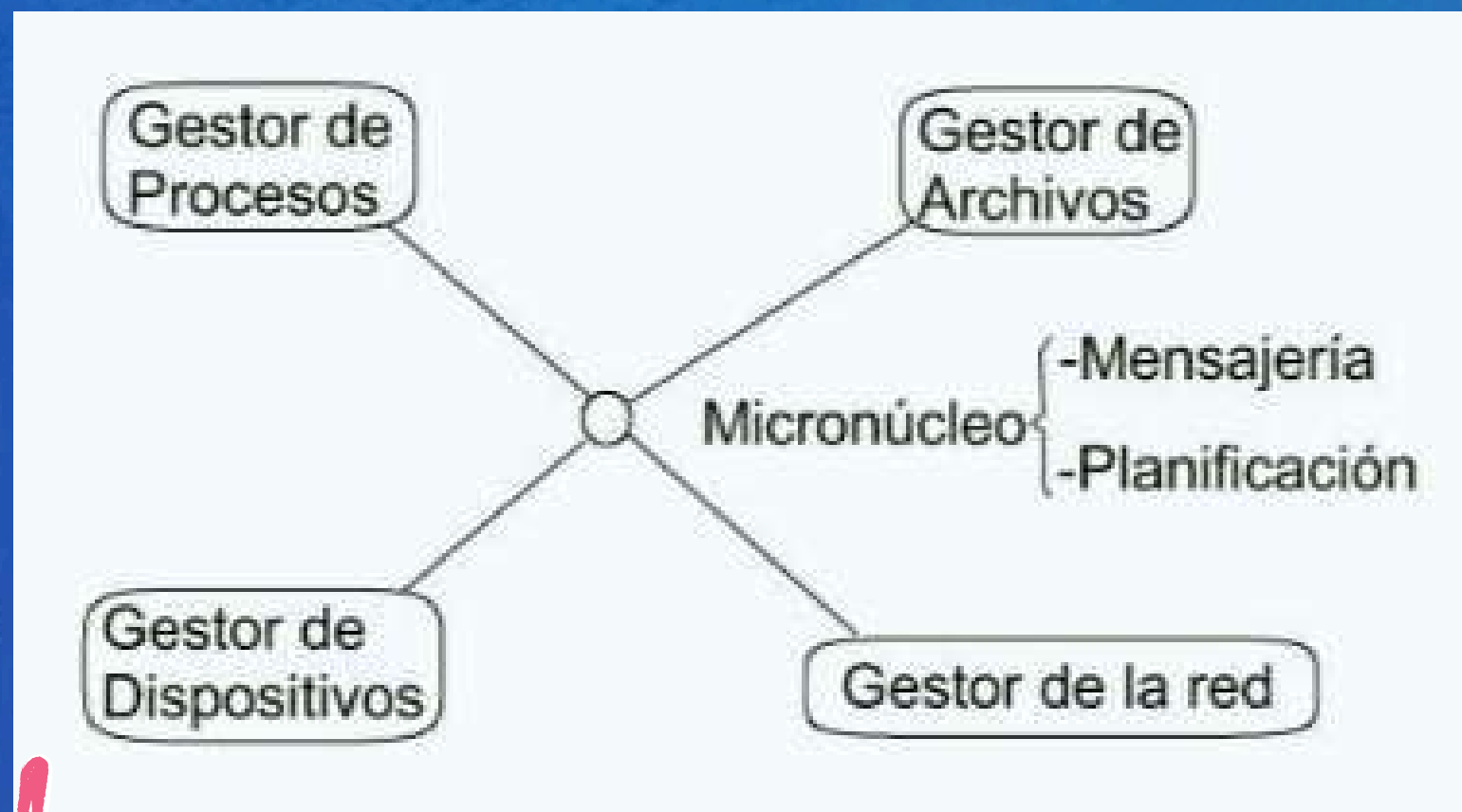
El sistema operativo QNX es un sistema operativo en tiempo real diseñado para tareas críticas. Sus puntos fuertes son el desarrollo de software, control de robots industriales y dispositivos embebidos.



Dan Dodge



Gordon Bell



QNX es un sistema operativo de tiempo real basado en Unix que cumple con la norma POSIX

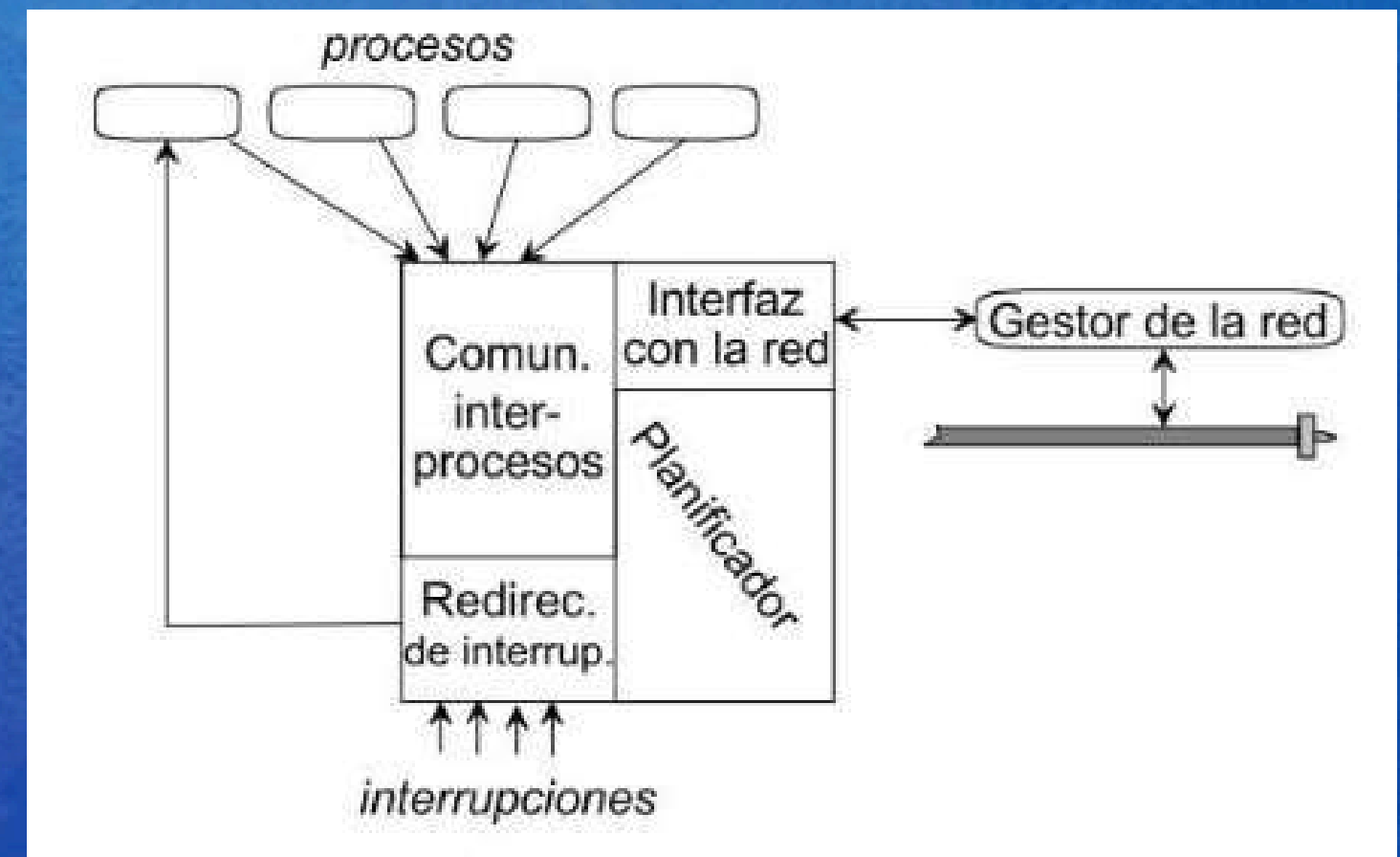
Características:

- Multitarea.
- Planificación de tareas expulsiva basada en prioridades.
- Fácilmente configurable y actualizable.
- Sencillez de estructura.
- Distributividad transparente implantada en el núcleo (red+intercomunicaciones).



Funciones del micronúcleo:

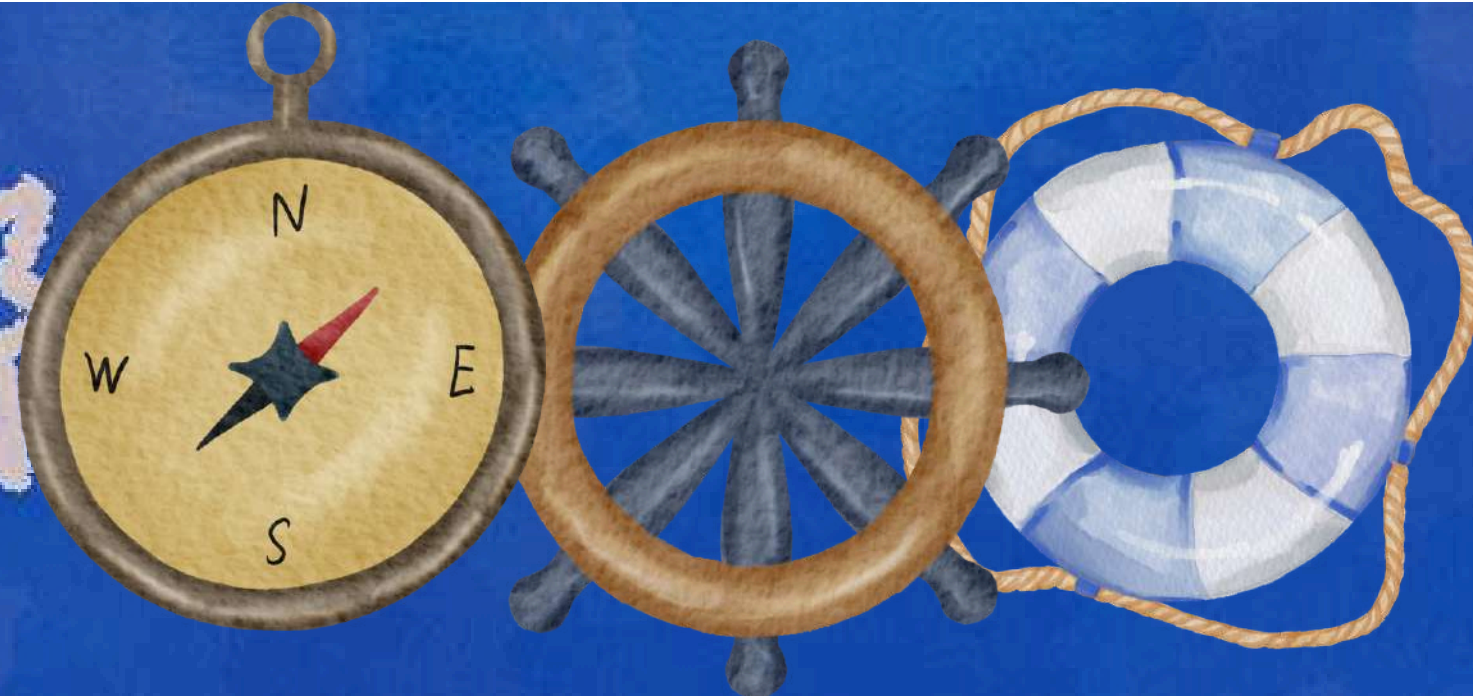
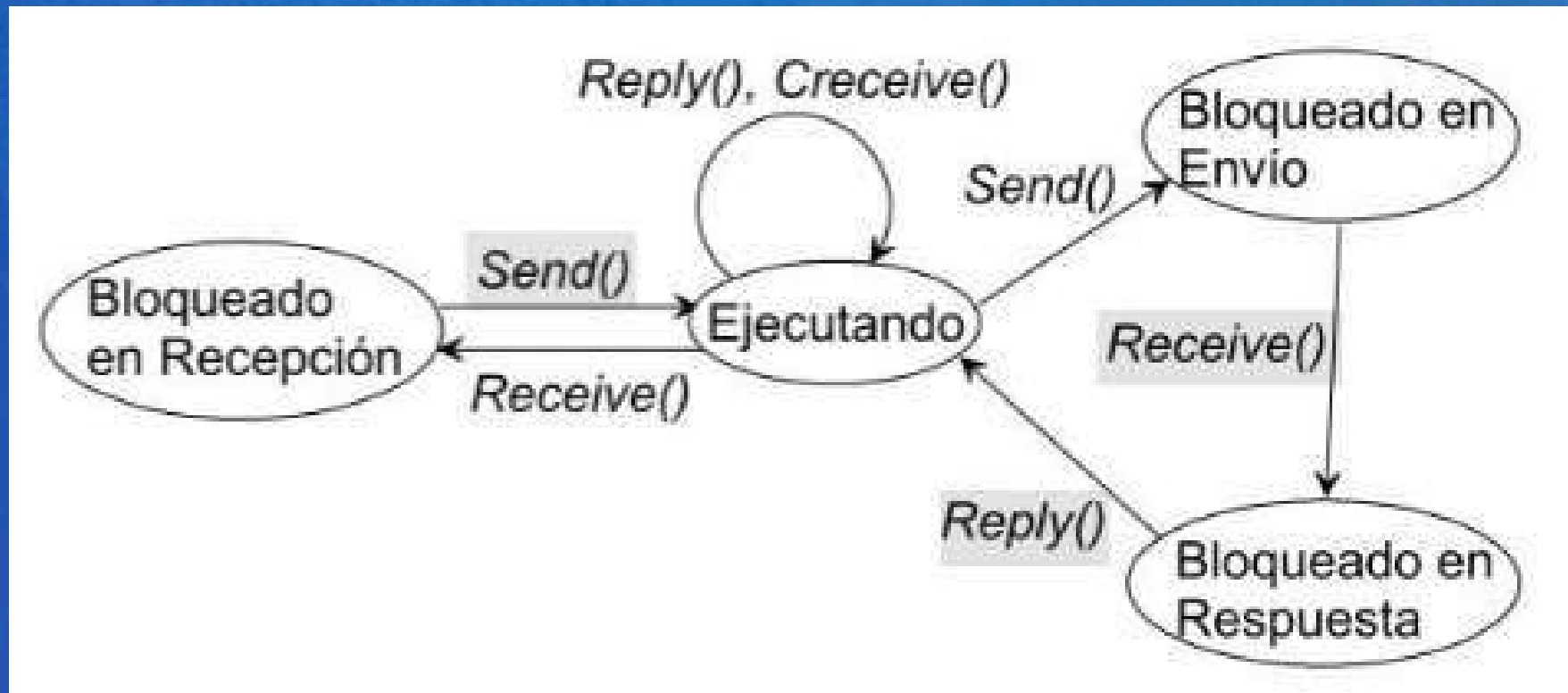
- Comunicación inter-procesos.
- Comunicación con la red.
- Planificación de procesos.
- Recepción y redirección de interrupciones.





Funciones de la comunicación inter-procesos:

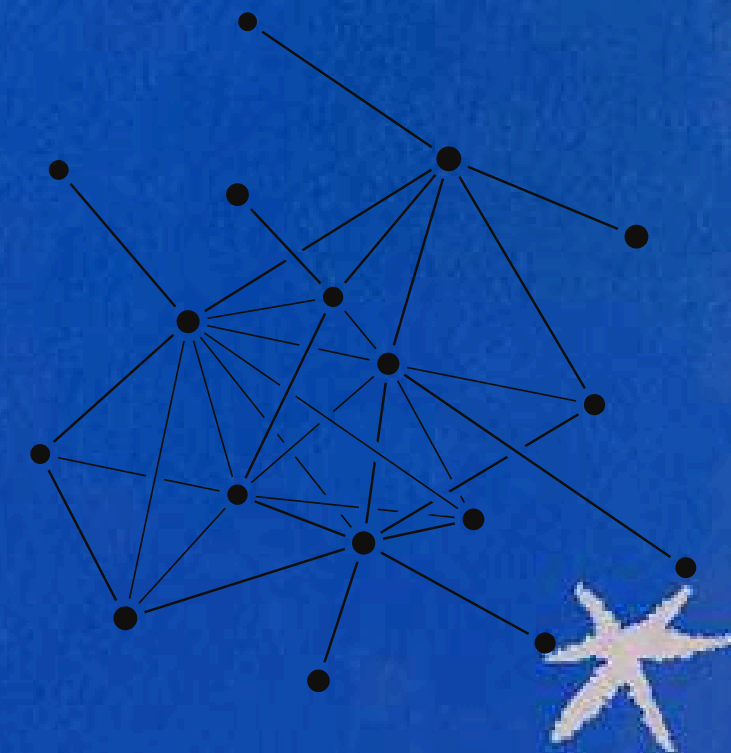
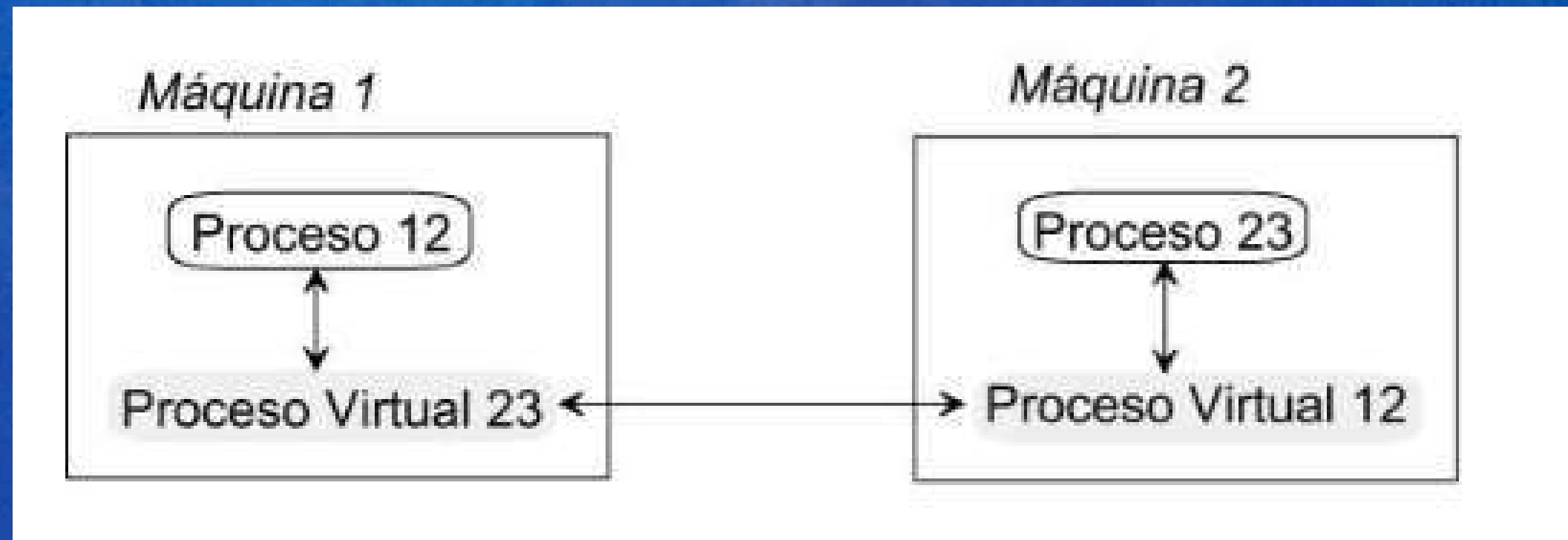
- Recepción Condicional (no bloqueante).
- Lectura/Escritura parciales de mensajes (para mensajes de longitud variable).
- Mensajes multipartitos (evitación de buffers intermedios de ensamblaje).
- "Proxies" (mensajes sin bloqueos y con intermediario).
- Señales (con acciones por defecto, ignorancia o manejadores, además de bloqueos).





Comunicacion con la RED.

- Círculos Virtuales:
 - Conexiones virtuales entre dos procesos.
 - Comunicaciones transparentes una vez el circuito virtual ha sido creado.
 - Robustez en las comunicaciones (detección de caídas de máquinas automática, detección de fallos en el cable automáticas, detección de caídas de procesos automáticas, etc.).
- Procesos Virtuales:
 - Imágenes de los extremos opuestos de los circuitos virtuales, en cada una de las máquinas.





Gestor de archivos:

Tipos de Ficheros gestionados por el gestor de archivos:

- Ficheros convencionales (de acceso aleatorio).
- Directorios (info sobre los ficheros que contienen).
- Enlaces Simbólicos (a ficheros de otros tipos).
- Tuberías ("pipes") y Colas (canales de comunicación entre procesos).
- Ficheros de bloques especiales (acceso a dispositivos hardware).

Protección de Ficheros

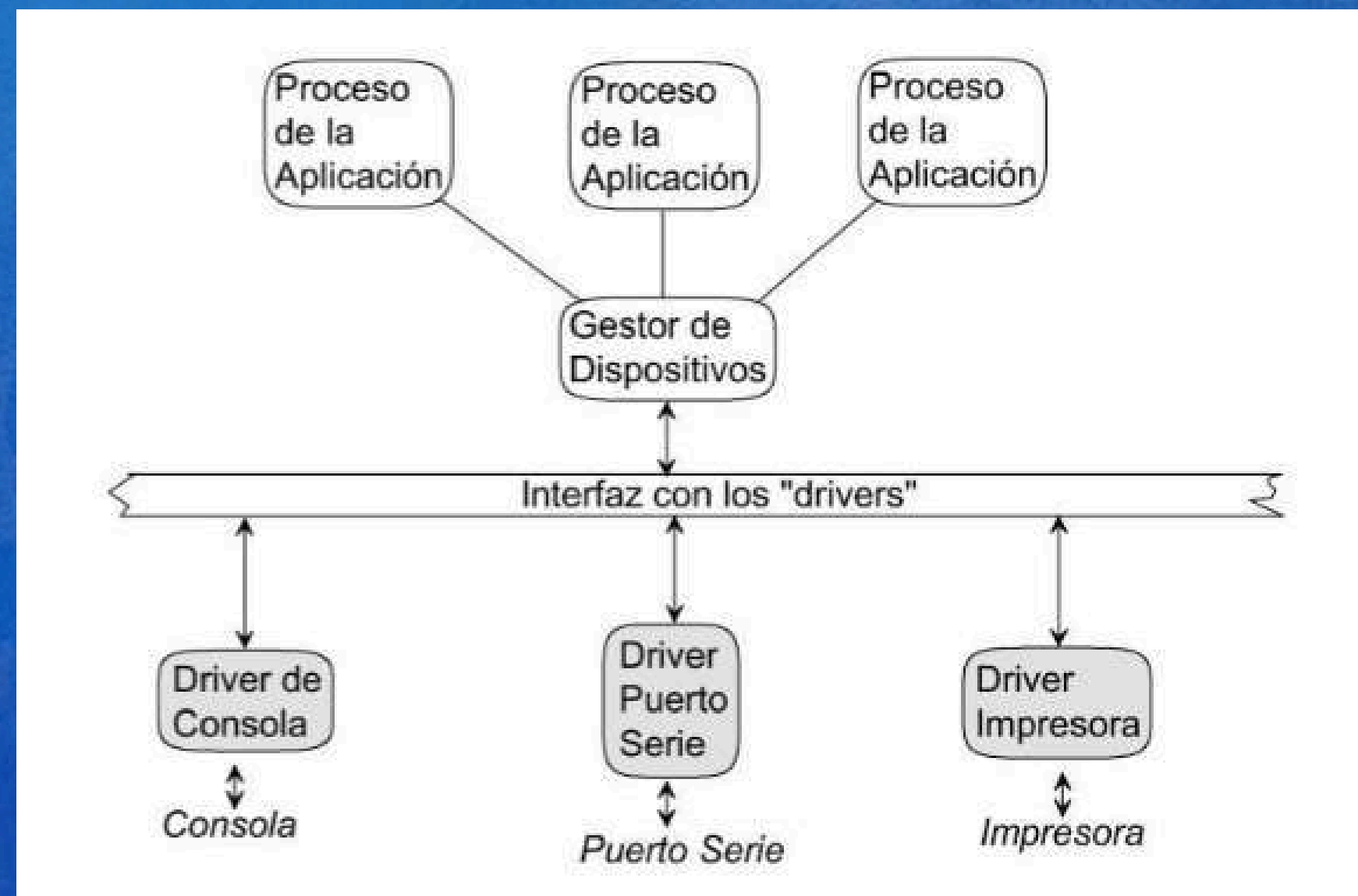
- Accesos de usuario.
- Accesos de grupo de usuarios.
- Accesos de otros usuarios.
- Lectura.
- Escritura.
- Ejecución.





Funciones del gestor de dispositivos:

- Gestion de las comunicaciones con los drivers de dispositivos hardware.





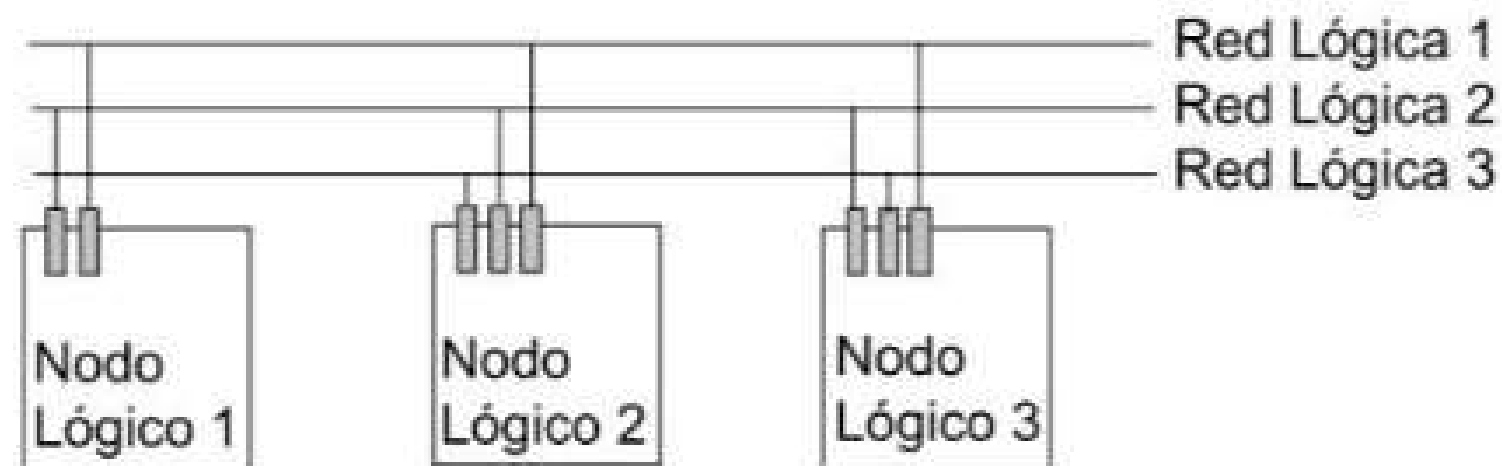
Funciones del gestor de la red:

- Gestión de las comunicaciones entre distintos ordenadores de la red.
- Balanceo de carga.
- Tolerancia a fallos vía conexiones redundantes.

Comunicaciones en Red:

- Toda la red se comporta para las aplicaciones como un único nodo.
- Una cola de futuras transmisiones conecta al gestor de la red con el micronúcleo y el gestor de procesos. Implementada en memoria compartida.
- Es independiente del hardware de red (tarjetas de red).

Redes y Nodos Lógicos para Balanceo de Carga y Tolerancia a Fallos:



Zephyr *SO*

Fue creado originalmente por Wind River Systems, una empresa conocida por desarrollar el sistema operativo en tiempo real VxWorks.

Actualmente, Zephyr es mantenido por una comunidad de desarrolladores y empresas patrocinadoras bajo la coordinación de la Linux Foundation, dentro de su iniciativa Zephyr Project.

Algunas de las empresas que han contribuido significativamente al desarrollo de Zephyr son:

- Intel
- NXP Semiconductors
- Nordic Semiconductor
- Google
- Texas Instruments
- Samsung

Zephyr OS

Arquitectura de Zephyr

Zephyr OS utiliza una arquitectura modular que optimiza memoria y energía al incluir solo componentes necesarios. Sus capas principales son:

- Kernel en Tiempo Real:
 - Gestiona tareas críticas con planificación, interrupciones y sincronización.
- Servicios de Sistema:
 - Incluye bibliotecas para gestión de memoria, archivos y E/S.



Zephyr OS

- Redes y Conectividad:
 - Soporta protocolos como IPv4/IPv6, Bluetooth, Zigbee, LoRa y MQTT para integración en IoT.
- Periféricos y Controladores:
 - Permite comunicación con hardware (sensores, actuadores, interfaces).
- Seguridad:
 - Ofrece aislamiento de procesos, criptografía y gestión de claves para protección contra ataques.



Zephyr SO

Ejemplo de caso practico de QNX:

QNX es un sistema operativo de tiempo real basado en microkernel.

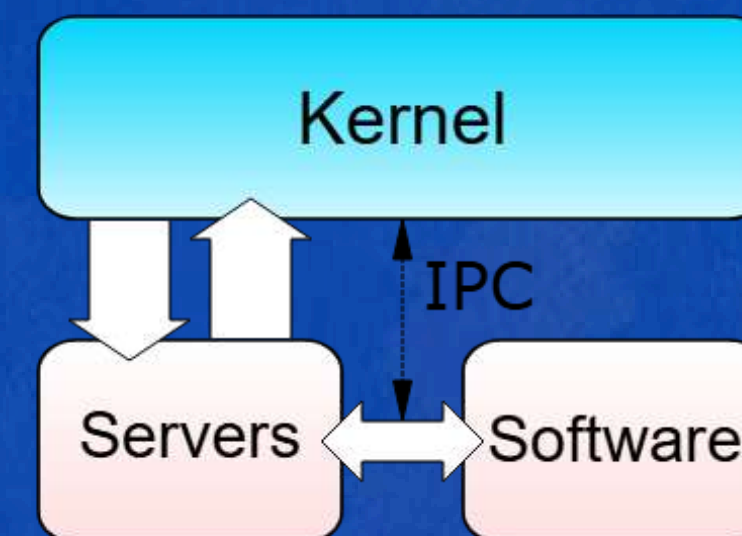
Esto significa que:

- El "núcleo" o microkernel de QNX hace solo tareas esenciales: administración de CPU, memoria, mensajes entre procesos e interrupciones.
- Todo lo demás (archivos, red, gráficos, etc.) corren como procesos separados, incluso los controladores de dispositivos.

- Todos los procesos se comunican entre sí mediante mensajes (esto se llama Inter Process Communication o IPC).

Esto hace que QNX nos permita:

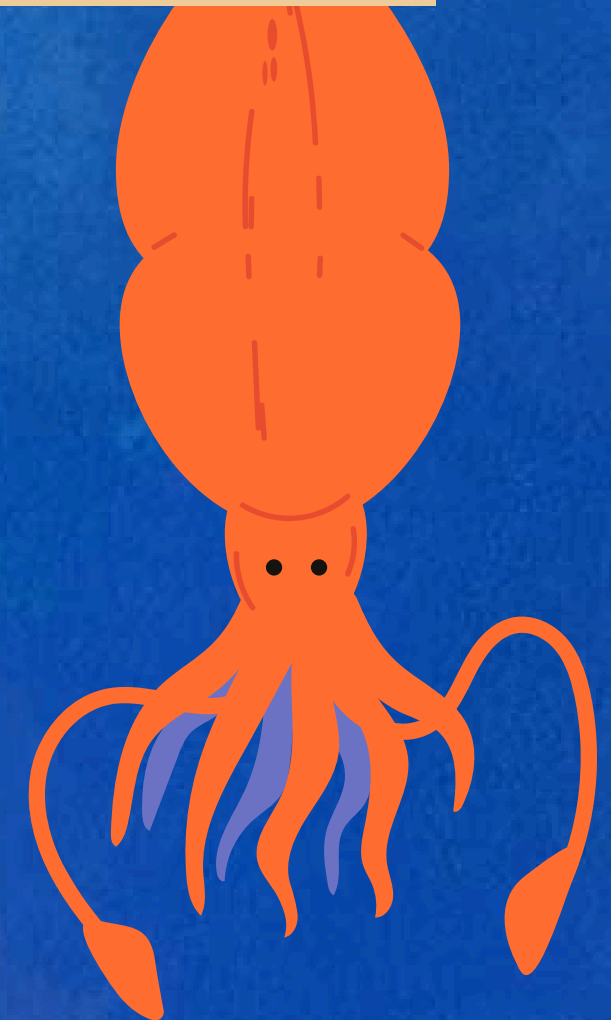
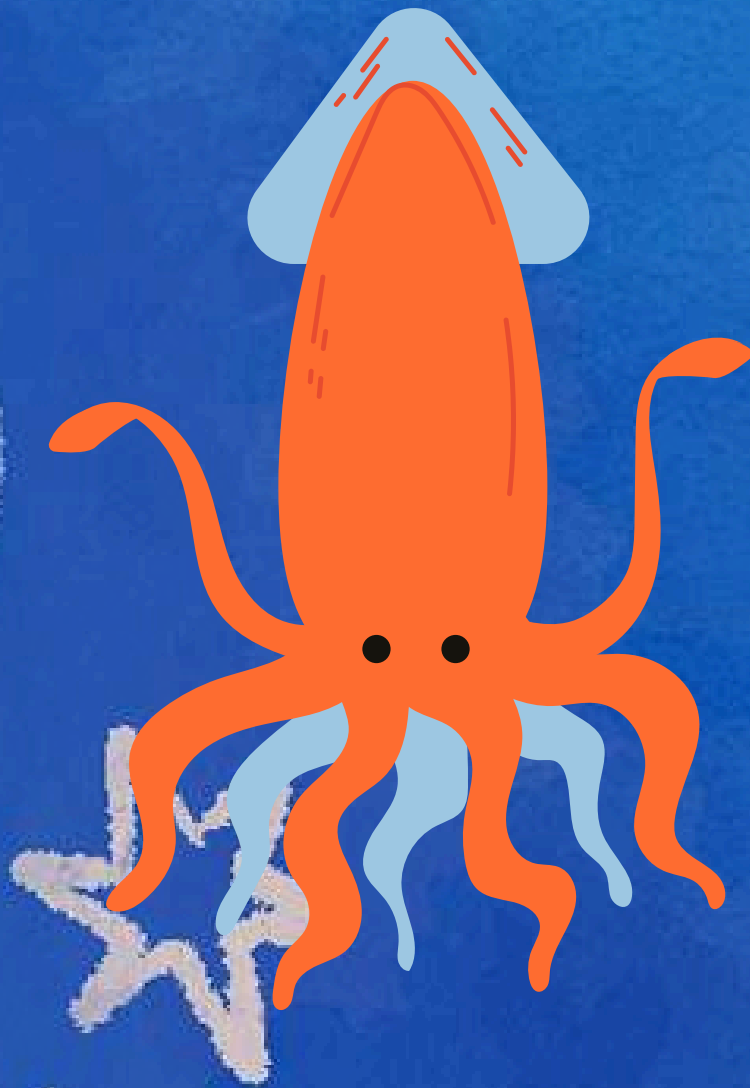
- Evitar la caída del sistema debido a un error en caso de que un servicio falle
- Modular: podemos agregar o quitar servicios fácilmente.
- Determinista: puede responder a eventos en tiempos precisos, esencial para navegación o sistemas de alerta.



Código Ejemplo: Comunicación entre Procesos en QNX

Código del servidor

```
1  #include <stdio.h>
2  #include <sys/neutrino.h>
3  #include <unistd.h>
4
5  int main() {
6      int chid;    // Channel ID
7      int rcvid;   // Receive ID
8      int temp = 85; // Temperatura simulada
9
10     // Crear un canal de comunicación
11     chid = ChannelCreate(0);
12     if (chid == -1) {
13         perror("Error al crear canal");
14         return 1;
15     }
16
17     printf("Servidor de temperatura en marcha. Esperando solicitudes...\n");
18
19     while (1) {
20         rcvid = MsgReceive(chid, NULL, 0, NULL); // Espera un mensaje
21         if (rcvid == -1) {
22             perror("Error al recibir mensaje");
23             continue;
24         }
25
26         // Responder al cliente con la temperatura
27         MsgReply(rcvid, 0, &temp, sizeof(temp));
28     }
29
30     return 0;
31 }
32
```



Código Ejemplo: Comunicación entre Procesos en QNX

```
#include <stdio.h>
#include <sys/neutrino.h>
#include <unistd.h>

int main() {
    int coid; // Connection ID
    int temp;

    // Conectarse al servidor (pid 1, canal 1 como ejemplo)
    coid = ConnectAttach(0, 1, 1, 0, 0);
    if (coid == -1) {
        perror("Error al conectar");
        return 1;
    }

    // Enviar una solicitud vacía y esperar la respuesta
    MsgSend(coid, NULL, 0, &temp, sizeof(temp));

    printf("Temperatura recibida del motor: %d°C\n", temp);

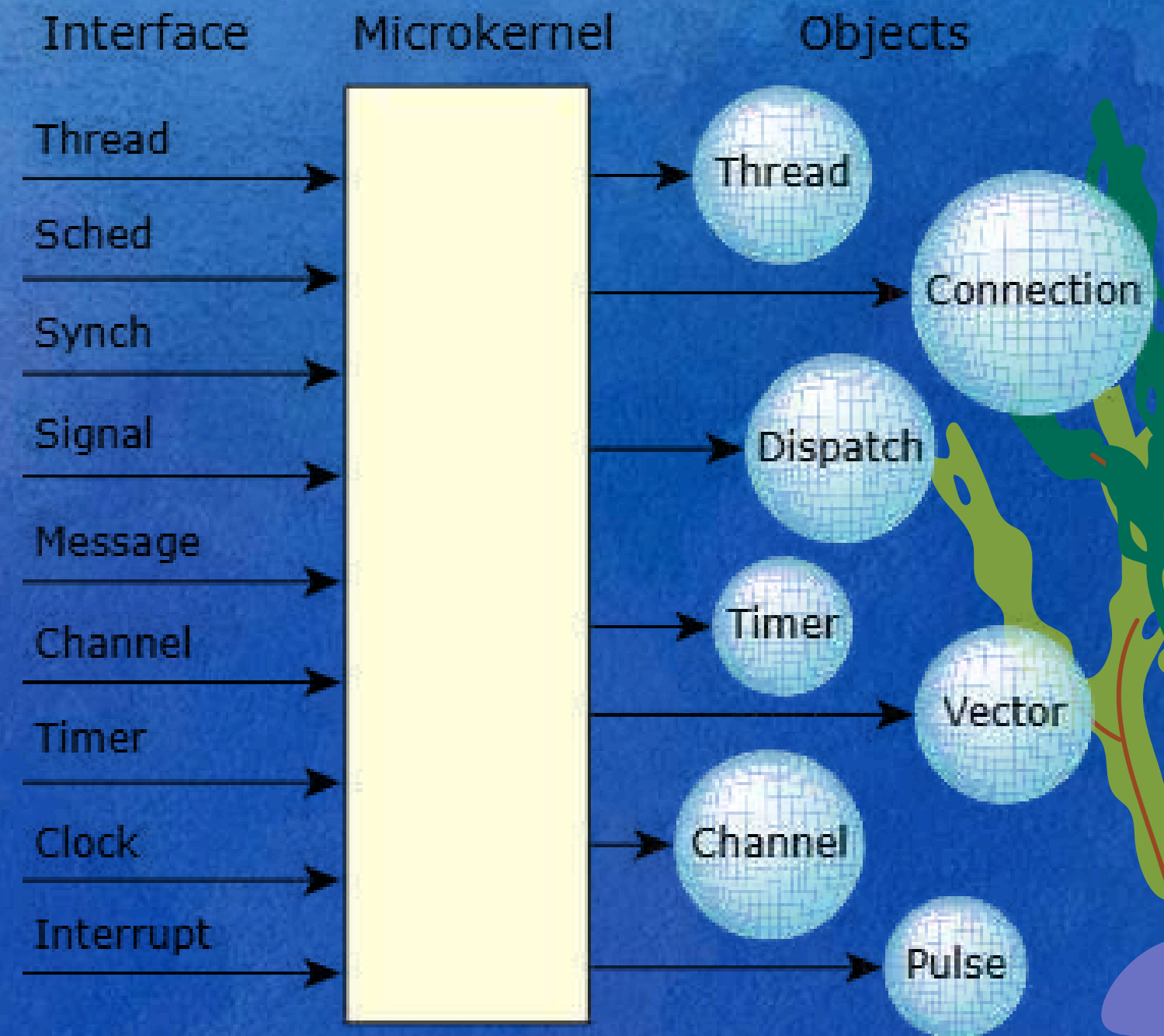
    return 0;
}
```

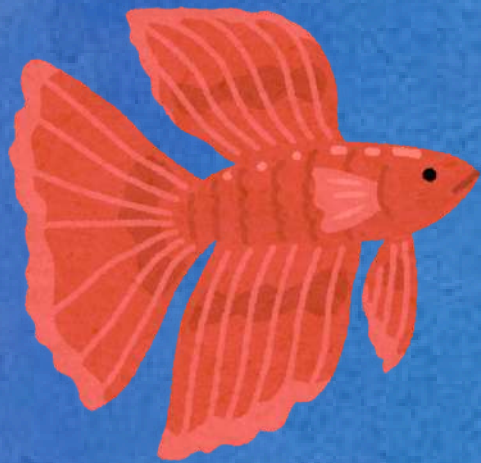
Código de cliente el cual solicita temperatura

Código Ejemplo: Comunicación entre Procesos en QNX

”En QNX, los procesos no se ven obligados a compartir memoria directamente. En su lugar, se envían mensajes entre sí de manera segura.

Por ejemplo, aquí vemos cómo un cliente solicita la temperatura del motor, y el servidor responde enviándole ese dato. Esto mantiene los procesos aislados y mejora la estabilidad y la seguridad del sistema.





¿Cómo funciona VxWorks en un caso práctico?

Todo en VxWorks corre como tareas.

Cada tarea tiene una prioridad: tareas más importantes (como alarmas) tienen prioridad máxima.

El SO planifica qué tarea corre en cada momento según su prioridad.

Usa semáforos para sincronizar acceso a recursos compartidos (ej., un radar o sensor).

```
## Starting application at 0x4010100000 ...

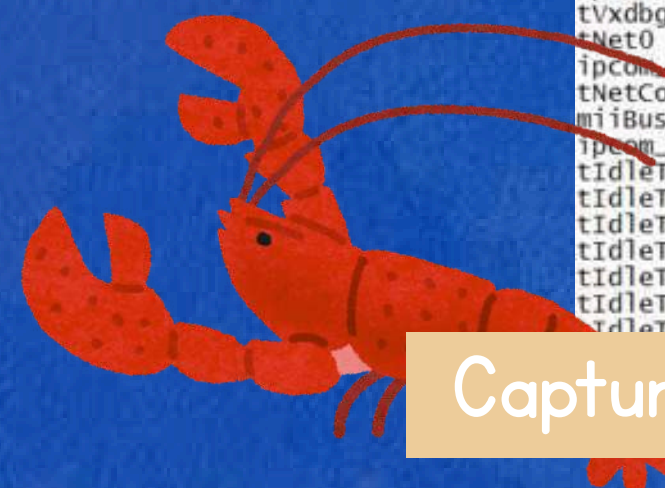
VxWorks 7 SMP 64-bit
Core Kernel version: 1.0.0.0
Build date: May 30 2014 10:51:05
Copyright Wind River Systems, Inc.
1984-2014

Board: Wind River Dev Kit MP8
CPU Count: 8
OS Memory Size: 1899MB
ED&R Policy Mode: Deployed

Adding 5290 symbols for standalone.
[vxworks]# i
```

NAME	TID	PRI	STATUS	PC	ERRNO	CPU #
tJobTask	40104cdbc0	0	PEND	401020c83c	0	-
tExcTask	40102a073c	0	PEND	401020c83c	0	-
tLogTask	40104d01d8	0	PEND	401020b0f0	0	-
tShell0	40105c1d30	1	READY	4010215e08	0	0
ipcom_tick>	401057a990	20	PEND	401020c83c	0	-
tvxdbgTask	401057dc20	25	PEND	401020c83c	0	-
tNet0	40104d3b78	50	PEND	401020c2b4	0	-
ipcom_sysl>	40104c9810	50	PEND	401020d3d4	0	-
tNetConf	40105a6e40	50	PEND	401020c83c	0	-
miibusMoni>	40104d5e08	252	DELAY	4010215640	0	-
ipcom_egd	4010583c20	255	DELAY	4010215640	0	-
tIdleTask0	40102a2fb0	287	READY	401020c004	0	-
tIdleTask1	40102a7220	287	READY	401020c00c	0	1
tIdleTask2	40102ab490	287	READY	401020c004	0	2
tIdleTask3	40102afb20	287	READY	401020c004	0	3
tIdleTask4	40102b1700	287	READY	401020c004	0	4
tIdleTask5	40102b2440	287	READY	401020c004	0	5
tIdleTask6	40102b4620	287	READY	401020c004	0	6
					0	7

Captura de arranque de VxWorks



Código ejemplo: Dos tareas en VxWorks sincronizadas con un semáforo

```
1  #include <vxWorks.h>
2  #include <taskLib.h>
3  #include <semLib.h>
4  #include <stdio.h>
5
6  SEM_ID semSensor; // Semáforo global
7  int sensorData = 0; // Dato leído
8
9  void sensorTask() {
10     while (1) {
11         // Simular lectura de sensor
12         sensorData = rand() % 100; // Valor entre 0-99
13         printf("Sensor: leído dato %d\n", sensorData);
14
15         semGive(semSensor); // Liberar semáforo para notificar a displayTask
16         taskDelay(100);      // Esperar 1 segundo (100 ticks)
17     }
18 }
19
20 void displayTask() {
21     while (1) {
22         semTake(semSensor, WAIT_FOREVER); // Esperar a que sensorTask lea
23         printf("Display: dato recibido: %d\n", sensorData);
24     }
25 }
26
27 void startTasks() {
28     // Crear semáforo binario
29     semSensor = semBCreate(SEM_Q_PRIORITY, SEM_EMPTY);
30
31     // Crear tareas
32     taskSpawn("tSensor", 100, 0, 2000, (FUNCPTR)sensorTask, 0,0,0,0,0,0,0,0,0,0);
33     taskSpawn("tDisplay", 90, 0, 2000, (FUNCPTR)displayTask, 0,0,0,0,0,0,0,0,0,0);
34 }
```

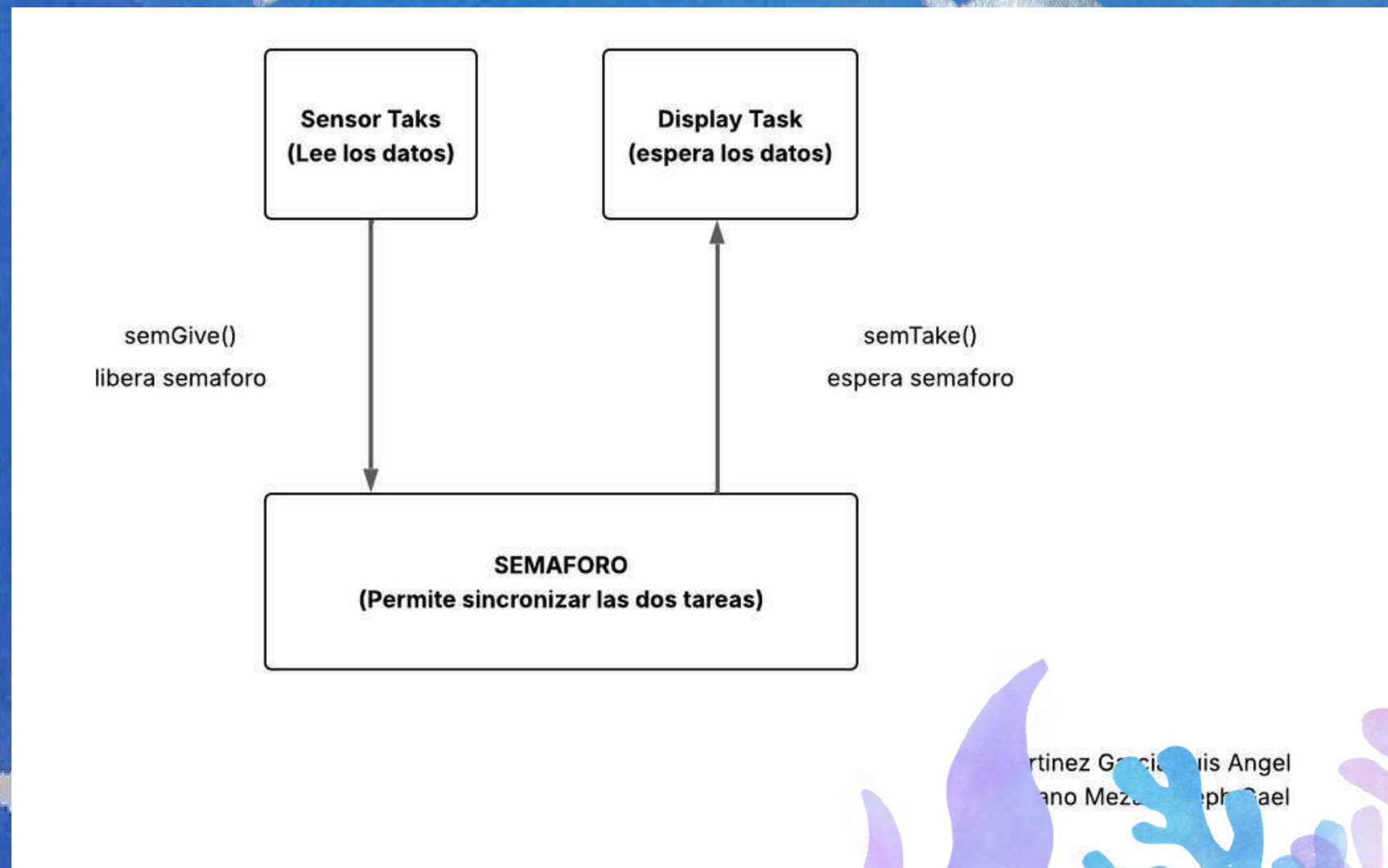
En VxWorks, el sistema gestiona tareas independientes que se ejecutan concurrentemente. Este ejemplo simula un sistema donde una tarea recolecta datos de un sensor, y otra tarea espera esos datos para procesarlos.

Ademas como vimos en clase, hicimos uso de un semáforo para sincronizar las tareas, asegurando que el sistema funcione correctamente en tiempo real.

Esto en un escenario es esencial para mantener la precisión y seguridad en algún equipo marítimo.

Diagrama de flujo: Comunicacion entre tareas

- SensorTask: Simula la lectura de un dato del sensor del barco.
- DisplayTask: Espera que SensorTask le notifique para mostrar los datos.
- Usamos un semáforo para coordinar la comunicación de manera segura.
- Beneficios:
 - Tareas críticas no se bloquean entre sí.
 - Asegura la respuesta inmediata en sistemas críticos.
 - Mantiene estabilidad y tiempo real, ideal para navegación y sistemas de control.



Estándares y Seguridad Marítima

Los sistemas operativos deben cumplir con normativas estrictas:

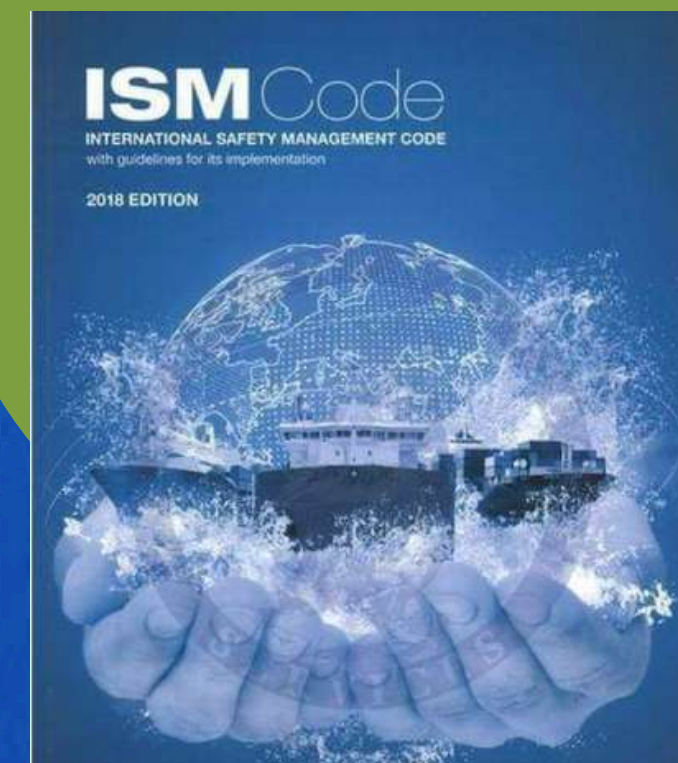
IMO (Organización Marítima Internacional).



SOLAS (Safety of Life at Sea).



ISM Code (International Safety Management).



Estándares de ciberseguridad como IMO MSC-FAL.1/Circ.3, que protege contra ciberataques a sistemas críticos.

Casos Reales y Tecnologías Emergentes

- QNX en barcos militares y cruceros: usado por su fiabilidad y modularidad.
- OpenCPN: sistema de navegación gratuito muy usado en embarcaciones de recreo.
- Automatización y barcos autónomos: Empresas como Rolls-Royce trabajan en buques sin tripulación, totalmente gestionados por SO inteligentes.
- IoT Marítimo: Conectividad entre sensores, motores y redes globales en tiempo real.

Retos y Futuro de los SO en la Industria Náutica

Retos

- Robustez en entornos extremos.
- Ciberseguridad marítima.
- Integración de sistemas heredados.
- Bajo consumo de recursos
- Comunicación y sincronización de sistemas distribuidos.
- Normativas y certificaciones.

Futuro

- Adopción de SO en tiempo real (RTOS) y sistemas embebidos avanzados.
- Inteligencia artificial y edge computing a bordo.
- Soporte para autonomía marítima.
- Virtualización y contenedores.
- Interoperabilidad en sistemas de flotas inteligentes.

Conclusión

Los sistemas operativos representan el núcleo funcional de los dispositivos tecnológicos que operan a bordo de embarcaciones. Gracias a ellos, se logra una coordinación eficiente entre los distintos elementos que forman parte del ecosistema náutico.

Con el avance continuo de la tecnología, los sistemas operativos se están transformando para integrar herramientas como la inteligencia artificial, la computación en el borde (edge computing) y la conectividad global de dispositivos. Esto implica no solo una mayor automatización, sino también la necesidad de mayor seguridad digital y flexibilidad en los sistemas.

Referencias

- <https://www.330ohms.com/blogs/blog/introduccion-a-los-so-en-tiempo-real-que-es-un-rtos>
- <https://compraco.com.br/es/blogs/tecnologia-e-desenvolvimento/como-o-zephyr-torna-o-desenvolvimento-de-software-embarcado-uma-brisa>
- <https://es.scribd.com/doc/142758972/QNX-Real-Time-Operating-System>
- <https://administraciondesistemas.com/zephyr-os-iot/>
- <https://www.yumpu.com/es/document/read/15668836/qnx-transparencias-en-pdf-departamento-de-ingenieria-de->
- <https://exponav.org/blog/puertos-y-buques/sistemas-de-control-en-buques/>
- <https://pronautictc.com/gps-nautico-uso-y-utilidades/>
- <https://azimutmarine.es/blog/post/sistema-de-posicionamiento-global-gps-historia-aplicacion-en-la-nautica.html>